Artificial Intelligence

BS (CS) Spring 2026

Lab 06 Manual



Learning Objectives:

1. Local Search Algorithm
2. Hill Climbing
3. Simulated Annealing
4. Local Beam Search
5. Genetic Algorithm

Lab Manual

Local Search Algorithm:

Local Search Algorithms are optimization techniques used in Artificial Intelligence to find solutions by iteratively exploring neighboring options in the solution space. That basically focus on iteratively **moving** from **one solution** to a **neighboring solution**. They do this by evaluating solutions based on a heuristic function, which provides a measure of the solution's quality. These algorithms aim to find the best solution in the vicinity of the current state, rather than exploring the entire solution space. This local exploration strategy is what makes them particularly valuable.
These algorithms start with an initial solution and make small, incremental changes to identify better alternatives. The primary goal is to achieve an optimal or satisfactory result **without exhaustively evaluating** the entire solution space.

Key Features of Local. Search Algorithms

1. **Objective Function:** Evaluates the quality of each solution to determine its suitability.
2. **Search Space:** Represents all possible solutions for the problem.
3. **Neighborhood Structure:** Defines the neighboring solutions accessible from a current solution.
4. **Evaluation Criteria:** Guides the selection of the next solution based on improvement or potential.

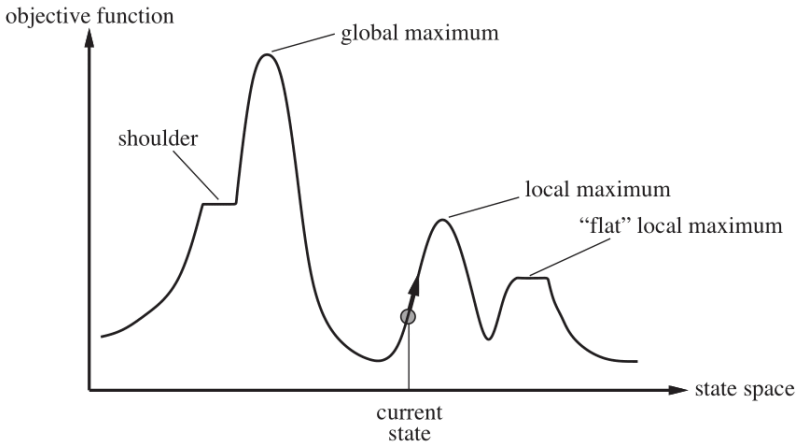


Figure 4.1 A one-dimensional state-space landscape in which elevation corresponds to the objective function. The aim is to find the global maximum. Hill-climbing search modifies the current state to try to improve it, as shown by the arrow. The various topographic features are defined in the text.

Local search algorithms allow us to find goal states without worrying about the path to get there. In local search problems, the state space consists of sets of "complete" solutions.

Hill Climbing

The Hill-Climbing Search Algorithm is a simple and widely used local search technique in Artificial Intelligence. It focuses on iteratively improving the current solution by moving to a neighboring solution with a higher objective function value. This process continues until no better neighboring solution is found, indicating that the algorithm has reached a peak or an optimal point.

How Hill-Climbing Works:

1. **Initialization:** Start with an initial solution in the search space.
2. **Evaluation:** Calculate the objective function value of the current solution.
3. **Neighbor Selection:** Move to a neighboring solution that provides the highest improvement.
4. **Termination:** Stop when no neighboring solution improves the current solution.

Pseudocode

```
function HILL-CLIMBING(problem) returns a state
    current ← make-node(problem.initial-state)
    loop do
        neighbor ← a highest-valued successor of current
        if neighbor.value ≤ current.value then
            return current.state
        current ← neighbor
```

Types of Hill-Climbing:

1. Simple Hill-Climbing: Evaluates one neighbor at a time and moves to the first better solution found.
2. Steepest-Ascent Hill-Climbing: Examines all neighbors and selects the best among them.
3. Stochastic Hill-Climbing: Randomly chooses a neighbor and moves to it if it improves the solution.

Advantages:

- Easy to implement and computationally efficient.
- Suitable for problems where evaluating all solutions is infeasible.

Limitations:

- Local Optima: May get stuck in suboptimal solutions if better neighbors are unavailable.
- Plateaus: Flat regions in the search space may slow progress.
- Ridges: Narrow peaks can be challenging to navigate.

Example:

Consider a robot trying to reach the highest point on a hill. Using the hill-climbing algorithm, the robot evaluates its current position and moves in the direction of increasing elevation until no further height can be gained.

Implementation

```python
def hill_climbing(initial_solution, objective_function):
    current_solution = initial_solution
    current_score = objective_function(current_solution)

    while True:
        neighbors = generate_neighbors(current_solution)
        best_neighbor = None
        best_neighbor_score = current_score

        for neighbor in neighbors:
            score = objective_function(neighbor)
            if score > best_neighbor_score:
                best_neighbor = neighbor
                best_neighbor_score = score

        if best_neighbor is None:
            break

        current_solution = best_neighbor
        current_score = best_neighbor_score

    return current_solution, current_score

def generate_neighbors(solution):
    # Example neighbor generation for a simple case
    return [solution + 1, solution - 1]

def objective_function(x):
    return -x**2  # Example: maximization problem

initial_solution = 0
best_solution, best_score = hill_climbing(initial_solution, objective_function)
print(f"Best solution: {best_solution} with score: {best_score}")
```

Simulated Annealing

Simulated Annealing is a local search algorithm inspired by the annealing process in metallurgy, where controlled cooling of a material results in a stable crystalline structure. This algorithm introduces randomness to overcome the limitations of traditional local search methods, such as getting stuck in local optima.

How Simulated Annealing Works:

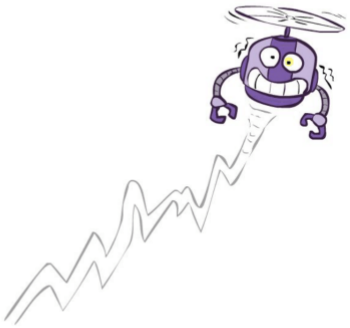
1. **Initialization:** Start with an initial solution and a high “temperature” parameter.
2. **Neighbor Selection:** Choose a random neighboring solution.
3. **Evaluation:** Compare the objective function values of the current and neighboring solutions.
   - If the new solution is better, move to it.

- If the new solution is worse, move to it with a probability that decreases as the "temperature" lowers.

4. **Cooling Schedule:** Gradually reduce the "temperature" over iterations, limiting the acceptance of worse solutions as the algorithm progresses.
5. **Termination:** Stop when the temperature reaches a predefined minimum or no significant improvement occurs.

Pseudocode

```
function SIMULATED-ANNEALING(problem,schedule) returns a state
current ← problem.initial-state
for t = 1 to ∞ do
    T ←schedule(t)
    if T = 0 then return current
    next ← a randomly selected successor of current
    ΔE ← next.value – current.value
    if ΔE > 0 then current ← next
            else current ← next only with probability e^(ΔE/T)
```



Key Features:

- **Escaping Local Optima**: The probabilistic acceptance of worse solutions helps avoid being trapped in local optima.
- **Controlled Exploration:** The "temperature" determines how likely worse solutions are accepted, balancing exploration and exploitation.

Advantages:

- Effective for problems with numerous local optima.
- Provides a mechanism for escaping plateaus and ridges in the search space.

Limitations:

- Performance depends on the cooling schedule and initial temperature.
- May require significant computational time for convergence.

Example Application:

In a scheduling problem, such as optimizing staff shifts, simulated annealing explores various schedules. Early iterations may accept suboptimal schedules to explore diverse possibilities, while later iterations focus on refining the solution.

Implementation

```python
import random
import math

def objective_function(route):
  # the total distance of the route
  return sum(math.sqrt((route[i] - route[i-1])**2) for i in range(len(route)))

def simulated_annealing(initial_route, temperature, cooling_rate):
  current_route = initial_route
  current_score = objective_function(current_route)
  best_route = current_route
  best_score = current_score

  while temperature > 0.1:
      new_route = current_route[:]
      i, j = random.sample(range(len(route)), 2)
      new_route[i], new_route[j] = new_route[j], new_route[i]
      new_score = objective_function(new_route)

      if new_score < current_score or random.random() < math.exp((current_score - new_score) / temperature):
          current_route = new_route
          current_score = new_score
          if new_score < best_score:
              best_route = new_route
              best_score = new_score

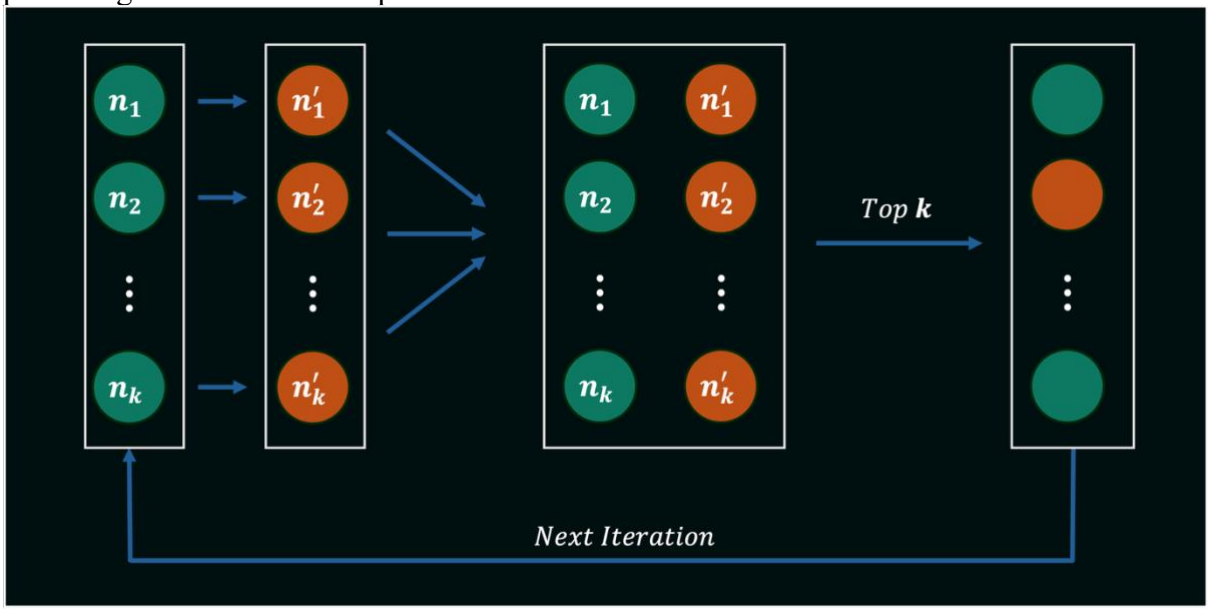
      temperature *= cooling_rate

  return best_route, best_score

# Example usage
route = [0, 1, 2, 3, 4]
best_route, best_score = simulated_annealing(route, 1000, 0.995)
print(f"Best route: {best_route} with score: {best_score}")
```

Local Beam Search

Local Beam Search is a local search algorithm that maintains multiple candidate solutions simultaneously, rather than focusing on a single solution like hill-climbing. It explores the solution space by analyzing the neighborhoods of all candidates and selects the most promising ones for further exploration.



How Local Beam Search Works:

1. **Initialization:** Begin with a set of k randomly selected candidate solutions.

2. **Neighbor Evaluation:** Evaluate the neighboring solutions of all candidates.
3. **Selection:** From the combined pool of neighbors and current candidates, select the top k solutions based on their objective function values.
4. **Iteration:** Repeat the process of exploring and selecting neighbors until convergence or a termination condition is met.

Pseudocode

```
function BEAM_SEARCH(start_node, goal_test, beam_width, max_depth):
    current_level ← [start_node]
    depth ← 0

    while current_level is not empty AND depth < max_depth:
        all_successors ← []

        // Generate all successors from current level
        for each node in current_level:
            if goal_test(node):
                return SUCCESS (node)

            successors ← get_successors(node)
            for each successor in successors:
                successor.score ← evaluation_function(successor)
                all_successors.append(successor)

        if all_successors is empty:
            return FAILURE

        // Sort all successors by their scores (best first)
        sorted_successors ← sort(all_successors, by=score, descending=True)

        // Keep only top beam_width candidates for next level
        current_level ← sorted_successors[0 : beam_width]
        depth ← depth + 1

    // Check final level for goal
    for each node in current_level:
        if goal_test(node):
            return SUCCESS (node)

    return FAILURE
```

Key Features:

- Parallel Exploration: By considering multiple candidates, it reduces the risk of getting stuck in local optima.
- Selective Focus: Focuses on the most promising areas of the solution space.

Advantages:

- Reduces the likelihood of being trapped in local optima compared to single-solution methods.

- Explores multiple paths simultaneously, increasing the chances of finding an optimal solution.

Limitations:

- Computationally intensive as it evaluates multiple candidates and their neighborhoods.
- The quality of results depends on the diversity of initial candidates.

Example:
In a route optimization problem, the algorithm starts with several different paths. It evaluates the neighborhoods of all paths and retains the best-performing ones, gradually converging to an optimal route.

Comparison with Hill-Climbing and Simulated Annealing:

- Unlike hill-climbing, local beam search evaluates multiple candidates simultaneously, avoiding reliance on a single solution.
- While simulated annealing incorporates randomness, local beam search systematically selects the best solutions at each step.